VLDB 2026 · BOSTON

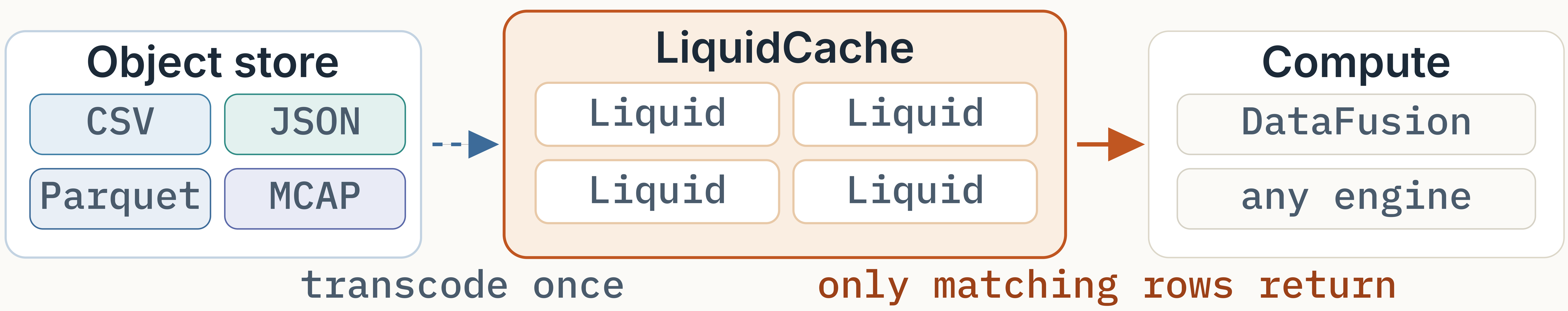
LiquidCache

Efficient Pushdown Caching for Cloud-Native Data Analytics

Xiangpeng Hao · Andrew Lamb (InfluxData) · Yibo Wu · Andrea Arpaci-Dusseau* · Remzi Arpaci-Dusseau*
University of Wisconsin–Madison · * joint senior authors

TAKEAWAY

Cache a better format than the storage format.



1 Why is a different format 10× cheaper?

- Writers choose storage formats for **stability and compatibility**, not query speed.
- Newer formats are faster to query: **10× less decoding CPU** than Parquet, with the same compression.
- Switching the lake to a new format **takes years**: re-encode petabytes, or convince every writer.
- Liquid lives **only inside the cache**: no old readers to support, so it can use the newest encodings.

2 Why is it zero overhead?

- Cloud analytics already runs a cache tier between the object store and compute. The server, its DRAM, and its network are **already paid for**.
- Serving reads is network- and memory-bound, so the cache's CPUs are **mostly idle**. Transcoding runs on those idle cores.
- Data is transcoded on the first miss, and only if a query touches it. Cold data is **never converted**.
- Transcoding overlaps the object-store fetch, so cold runs match Parquet; warm runs are **2× faster**.

RESULTS

10×	4×	100×	2×
lower CPU per query	**smaller than Arrow**	**less network traffic**	**faster warm queries**
not end-to-end latency	in memory, query-ready	filters run in the cache	cold runs match Parquet

ClickBench and TPC-H on Apache DataFusion, vs. non-pushdown caches · CloudLab 6525: 16 cores, 128 GB RAM, 10 Gbps.
Techniques upstreamed to **Apache Arrow**, **Parquet**, and **DataFusion**.

SCAN FOR THE TALK
liquid-cache-vldb.xiangpeng.systems
github.com/XiangpengHao/liquid-cache

SUPPORTED BY
influxdata · bauplan · spiral